COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	JULY 21 2026	**DATE SUBMITTED:**	JULY 27 2026
NAME:	DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 3.1 **Hands-on Activity 3.1 Linked Lists**

1. PROCEDURE OUTPUT

ILO A: Construct C++ code for a singly and doubly linked list in C++

Screenshot

Code:

```cpp
#include <iostream>
#include "simply_LL.h"

int main(){
    //creating an empty list
    SimplyList<char>* head = new SimplyList<char>;
    SimplyList<char>* second = new SimplyList<char>;
    SimplyList<char>* third = new SimplyList<char>;
    SimplyList<char>* fourth = new SimplyList<char>;
    SimplyList<char>* fifth = new SimplyList<char>;
    SimplyList<char>* sixth = new SimplyList<char>;

    // assign the data
    head->data = 'C';
    head->next = second;

    second->data = 'P';
    second->next = third;

    third->data = 'E';
    third->next = fourth;

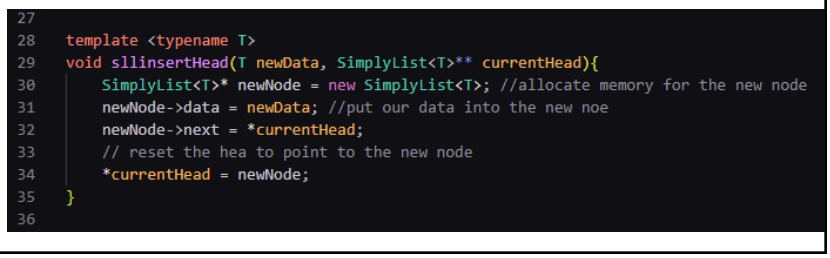
    fourth->data = '0';
    fourth->next = fifth;

    fifth->data = '1';
    fifth->next = sixth;

    sixth->data = '0';
    sixth->next = nullptr;
```

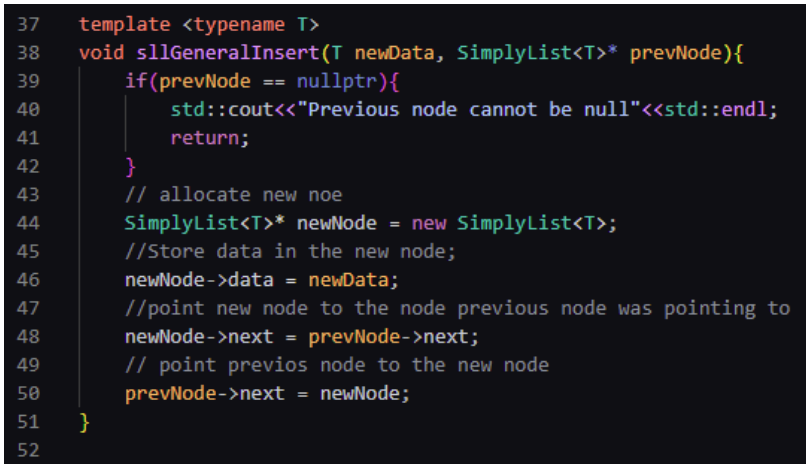
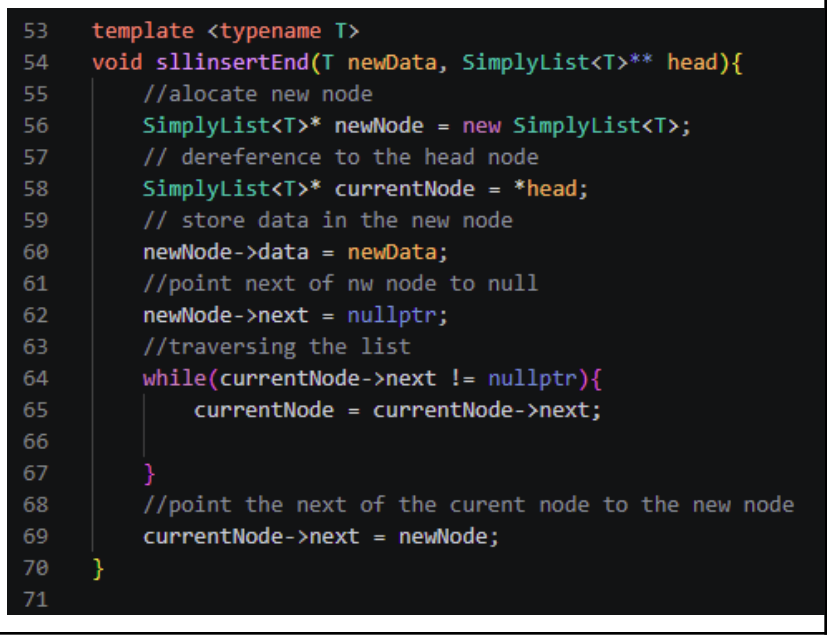
	Output: C:\Users\Dread\Desk Original list C->P->E->0->1->0
Discussion	The program created a simple implementation of linked lists. Each of the nodes were assigned a single character from the head to the fourth node, which are C, P, E, 0, 1, AND 0 and after that the program will display the data inside those nodes.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	
Insertion at head	

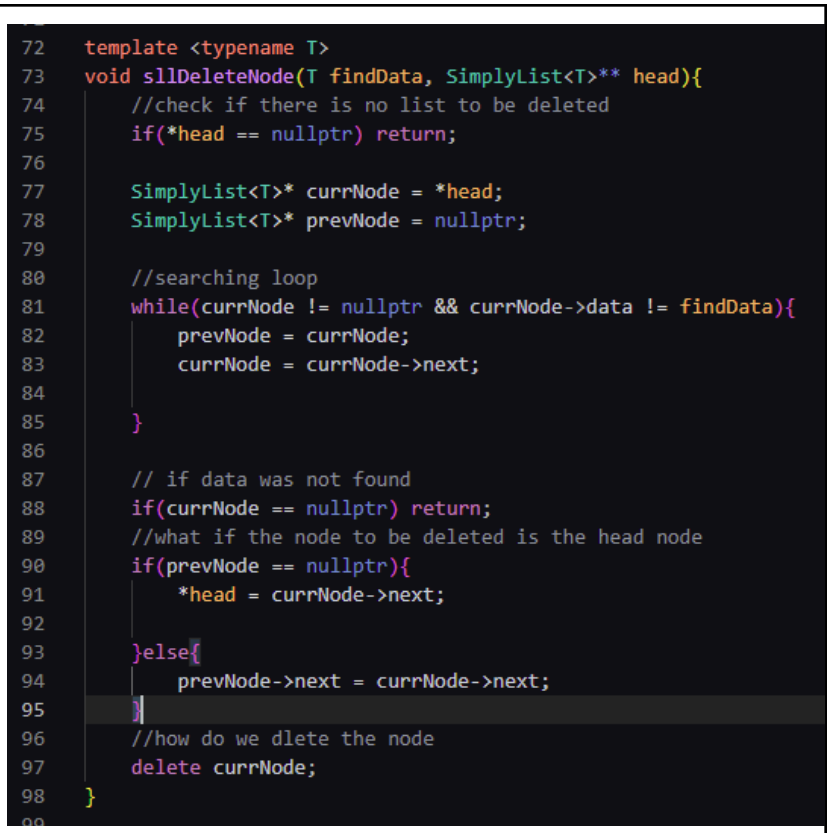
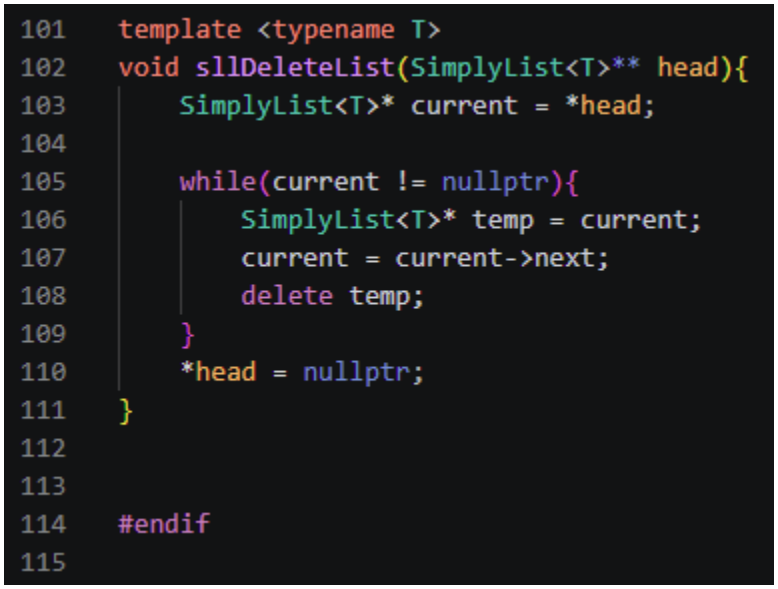
```
11  template <typename T>
12  void listTraversal(SimplyList<T>* head){
13      if(head == nullptr){
14          std::cout<<"List is empty"<<std::endl;
15          return;
16      }
17
18      while(head != nullptr){
19          std::cout<<head->data;
20          if(head->next != nullptr){
21              std::cout<<"->";
22          }
23          head = head->next;
24      }
25      std::cout<<std::endl;
26  }
27
```

```
27
28  template <typename T>
29  void sllinsertHead(T newData, SimplyList<T>** currentHead){
30      SimplyList<T>* newNode = new SimplyList<T>; //allocate memory for the new node
31      newNode->data = newData; //put our data into the new noe
32      newNode->next = *currentHead;
33      // reset the hea to point to the new node
34      *currentHead = newNode;
35  }
36
```

Insertion at any part of the list	
Insertion at the end	

```
37  template <typename T>
38  void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
39      if(prevNode == nullptr){
40          std::cout<<"Previous node cannot be null"<<std::endl;
41          return;
42      }
43      // allocate new noe
44      SimplyList<T>* newNode = new SimplyList<T>;
45      //Store data in the new node;
46      newNode->data = newData;
47      //point new node to the node previous node was pointing to
48      newNode->next = prevNode->next;
49      // point previos node to the new node
50      prevNode->next = newNode;
51  }
52
```

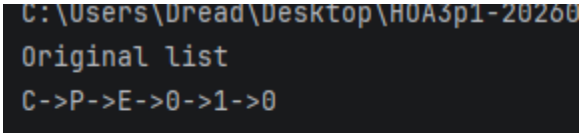
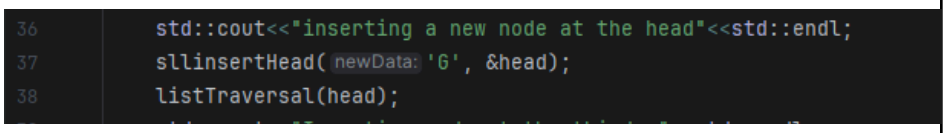
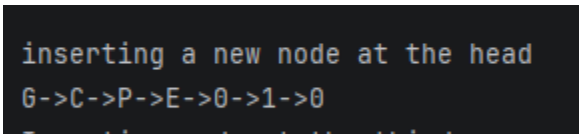
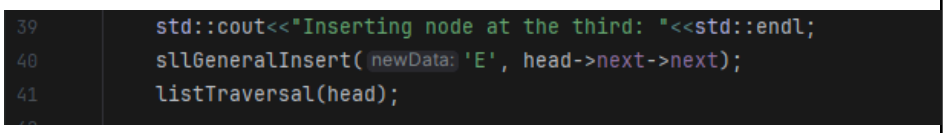
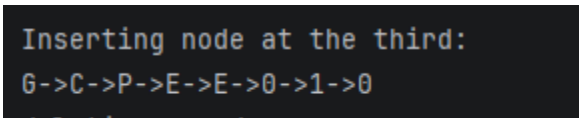
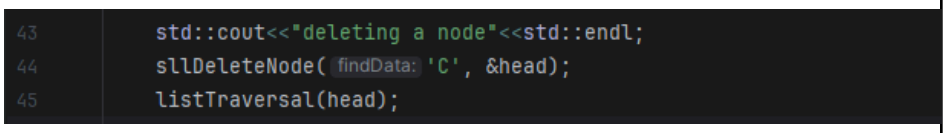
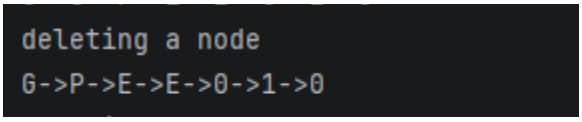
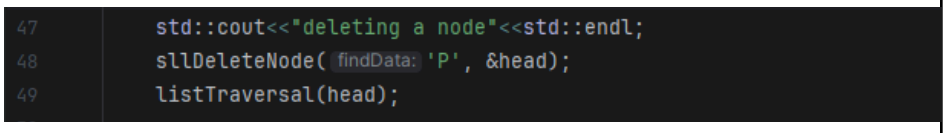
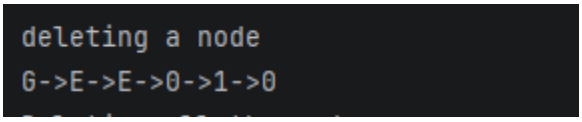
```
53  template <typename T>
54  void sllinsertEnd(T newData, SimplyList<T>** head){
55      //alocate new node
56      SimplyList<T>* newNode = new SimplyList<T>;
57      // dereference to the head node
58      SimplyList<T>* currentNode = *head;
59      // store data in the new node
60      newNode->data = newData;
61      //point next of nw node to null
62      newNode->next = nullptr;
63      //traversing the list
64      while(currentNode->next != nullptr){
65          currentNode = currentNode->next;
66
67      }
68      //point the next of the curent node to the new node
69      currentNode->next = newNode;
70  }
71
```

Deletion of a node	
Delete all list	

```
72  template <typename T>
73  void sllDeleteNode(T findData, SimplyList<T>** head){
74      //check if there is no list to be deleted
75      if(*head == nullptr) return;
76
77      SimplyList<T>* currNode = *head;
78      SimplyList<T>* prevNode = nullptr;
79
80      //searching loop
81      while(currNode != nullptr && currNode->data != findData){
82          prevNode = currNode;
83          currNode = currNode->next;
84
85      }
86
87      // if data was not found
88      if(currNode == nullptr) return;
89      //what if the node to be deleted is the head node
90      if(prevNode == nullptr){
91          *head = currNode->next;
92
93      }else{
94          prevNode->next = currNode->next;
95      }
96      //how do we dlete the node
97      delete currNode;
98  }
```

```
101 template <typename T>
102 void sllDeleteList(SimplyList<T>** head){
103     SimplyList<T>* current = *head;
104
105     while(current != nullptr){
106         SimplyList<T>* temp = current;
107         current = current->next;
108         delete temp;
109     }
110     *head = nullptr;
111 }
112
113
114 #endif
115
```

Table 3-2. Code for the List Operations

a.	Source Code	
	Console	
b.	Source Code	
	Console	
c.	Source Code	
	Console	
d.	Source Code	
	Console	
e.	Source Code	
	Console	

f.	Source Code	`std::cout<<"Final list"<<std::endl;` `listTraversal(head);`
	Console	`Final list` `G->E->E->0->1->0`

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots(s)	**Analysis**

`#ifndef DOUBLE_LL_H`
`#define DOUBLE_LL_H`
 	
`template <typename T>`
`class DoubleList{`
`public:`
`T data;`
`DoubleList<T>* prev = nullptr;`
`DoubleList<T>* next = nullptr;`
 	
`};`
	This block of code essentially defines a template class named DoubleList. Furthermore, it acts as a node for a doubly linked list and the next and prev pointers are initialized to nullptr so that it can link the adjacent elements.

`template <typename T>`
`DoubleList<T>* createNode(T newData) {`
`//dynamically allocate memory`
`DoubleList<T>* newNode = new DoubleList<T>;`
 	
`newNode->data = newData;`
`newNode->prev = nullptr;`
`newNode->next = nullptr;`
 	
`//return the address`
`return newNode;`
`}`
	This dynamically allocates memory on the heap for a new node while assigning its prev and next pointers to nullptr, then returns the created node.

```cpp
template <typename T>
void dllTraversal(DoubleList<T>* currentNode) {

    DoubleList<T>* tail;
    //checking if the list empty
    if (currentNode == nullptr) {
        std::cout<<"Empty List"<<std::endl;
    }

    //forwards traversing:
    std::cout<<"Forward: "<<std::endl;
    //moves towards the last node(till it becomes pointer)
    while (currentNode != nullptr) {
        std::cout<<currentNode->data<<" ";
        //saved the current node as the last visited noted
        tail = currentNode;
        //move to the next node
        currentNode = currentNode->next;

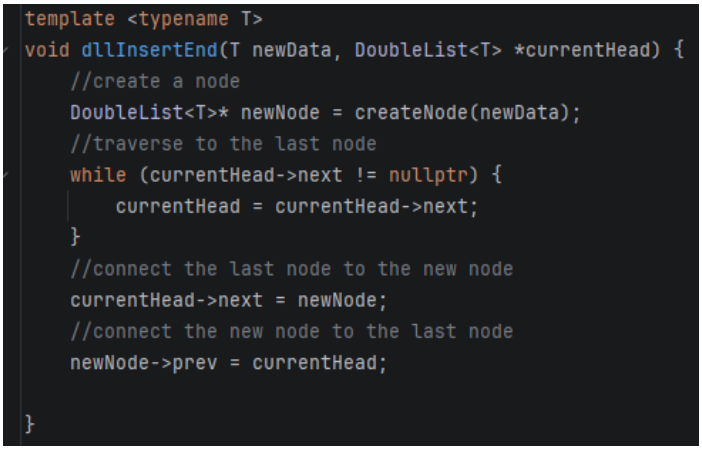
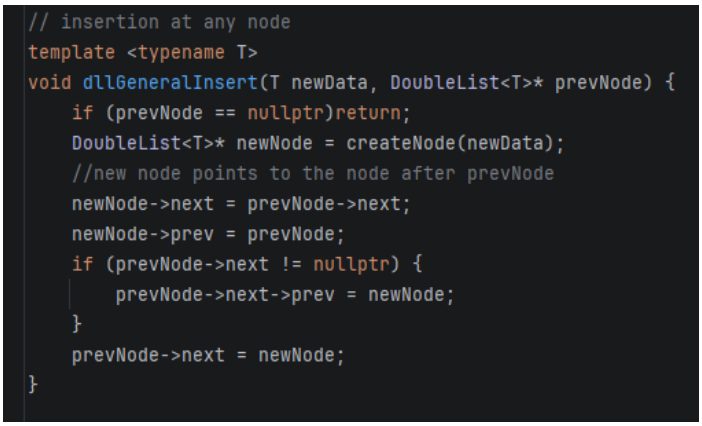
    }
    std::cout<<""<<std::endl;
    //backward traversing
    std::cout<<"Backward: "<<std::endl;
    //move toward the head
    while (tail != nullptr) {
        //printed the data
        std::cout<<tail->data<<" ";
        //move towards the previous node
        tail = tail->prev;
    }
    std::cout<<std::endl;

}
```

This function traverses the list, so it displays every data on each node. Moreover, it has a forward traversal which begins from the head. And a backwards traversal which begins from the tail. But if the currentNode is equivalent to nullptr it will display the list as empty.

```cpp
template <typename T>
void dllInsertHead(T newData, DoubleList<T>** currentHead) {
    //create a node
    DoubleList<T>* newNode = createNode(newData);
    //new node should point to the current head
    newNode->next = *currentHead;
    //current head should point back to the new node
    (*currentHead)->prev = newNode;
    //update the header pointer
    *currentHead = newNode;
}
```

This function inserts a new node that contains newData at the head. After that, it updates the pointer so that the new node links to the current head and the prev pointer of the old head is pointed back to the new node, then the currentHead is updated to point to the new node.

	This function inserts a new node that contains newData at the end. It traverses the list to find the last node and update the next and prev pointers to attach the new node to the tail of the list.
	This function inserts a new node that contains newData. It updates the prev and next pointers of the node that is near where the new node will be inserted.

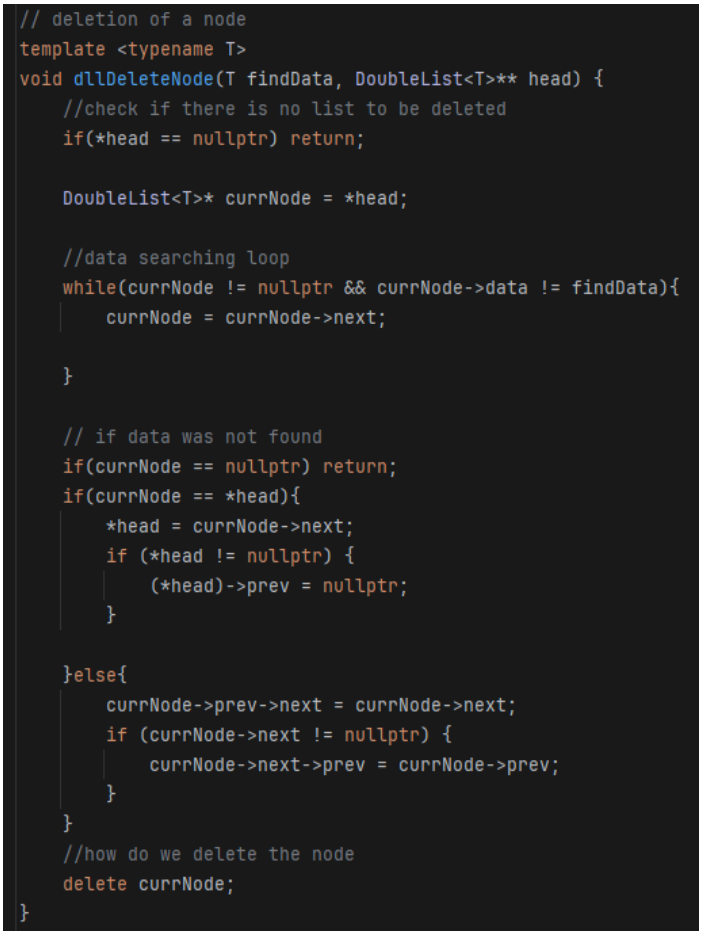
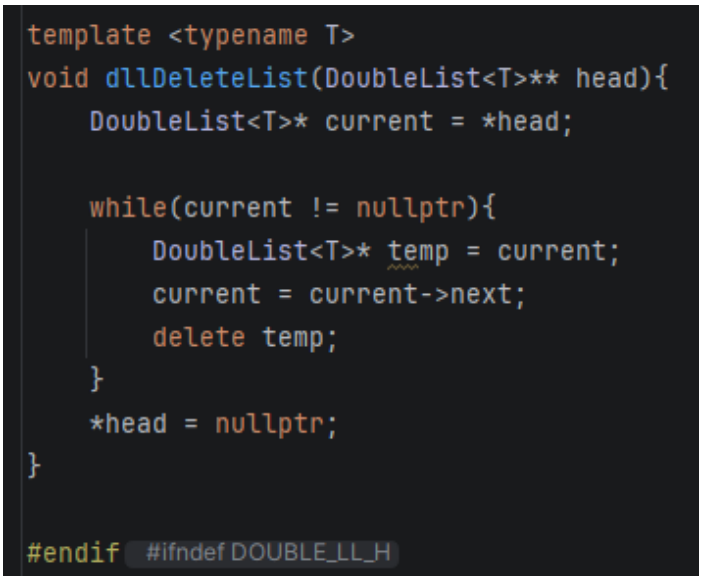
	This function deletes a node, it searches for the data that is to be deleted and when it finds it, it will delete the data and the node. When a node is deleted, it updates the pointers of the previous node to point to the next node. And when the head is deleted, the next node is set to be the new head.
	This function deletes the whole list. Moreover, it deallocates all memory allocated for the doubly linked list and then points it to null.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Header file:

```
1
2   #ifndef CIRCULAR_DLL_H
3   #define CIRCULAR_DLL_H
4
5   template <typename T>
6   class DoubleList{
7       public:
8       T data;
9       DoubleList<T>* prev = nullptr;
10      DoubleList<T>* next = nullptr;
11
12  };
13
14  // create a new node;
15
16  template <typename T>
17  DoubleList<T>* createNode(T newData) {
18      //dynamically allocate memory
19      DoubleList<T>* newNode = new DoubleList<T>;
20
21      newNode->data = newData;
22      newNode->prev = nullptr;
23      newNode->next = nullptr;
24      //return the address
25      return newNode;
26  }
```

```cpp
template <typename T>
void cdllTraversal(DoubleList<T>* head) {
    //checking if the list empty
    if (head == nullptr) {
        std::cout<<"Empty Playlist"<<std::endl;
        return;
    }
    DoubleList<T>* current = head;
    do {
        std::cout<<current->data<<"->";
        current = current->next;

    }
    while (current != head);
    std::cout<<"Playlist Restarted"<<std::endl;

}

template <typename T>
void cdllInsertEnd(T newData, DoubleList<T>** head) {
    DoubleList<T>* newNode = createNode(newData);
    if (*head == nullptr) {
        *head = newNode;
        newNode->next = *head;
        newNode->prev = *head;
        return;

    }
    DoubleList<T>* tail = (*head)->prev;
    newNode->next = *head;
    newNode->prev = tail;
    tail->next = newNode;
    (*head)->prev = newNode;

}
```

```cpp
template <typename T>
void cdllDeleteNode(T findData, DoubleList<T>** head) {
    if (*head == nullptr) return;
    DoubleList<T>* currNode = *head;

    do {
        if (currNode->data == findData) break;
        currNode = currNode->next;
    }
    while (currNode != *head);

    if (currNode->data != findData) return;

    if (currNode->next == currNode) {
        delete currNode;
        *head = nullptr;
        return;
    }

    if (currNode == *head) {
        *head = currNode->next;
    }

    currNode->prev->next = currNode->next;
    currNode->next->prev = currNode->prev;
    delete currNode;
}

```

```cpp
template <typename T>
void cdllDeleteList(DoubleList<T>** head) {
    if (head == nullptr || *head == nullptr) return;
    DoubleList<T>* current = *head;
    (*head)->prev->next = nullptr;

    while (current != nullptr) {
        DoubleList<T>* temp = current;
        current = current->next;
        delete temp;
    }
    *head = nullptr;
}

template <typename T>
void next(DoubleList<T>** currTrack) {
    if (currTrack == nullptr || *currTrack == nullptr){
        std::cout<<"Empty Playlist"<<std::endl;
        return;
    }
    *currTrack = (*currTrack)->next;
    std::cout<<(*currTrack)->data<<" is now playing"<<std::endl;
}

template <typename T>
void previous(DoubleList<T>** currTrack) {
    if (currTrack == nullptr || *currTrack == nullptr) {
        std::cout<<"Empty Playlist"<<std::endl;
        return;
    }
    *currTrack = (*currTrack)->prev;
    std::cout<<(*currTrack)->data<<" is now playing"<<std::endl;
}

#endif #ifndef CIRCULAR_DLL_H
```

Main code:

```cpp
#include <iostream>
#include <string>
#include "circular_DLL.h"

int main() {

    DoubleList<std::string>* playlist = nullptr;
    DoubleList<std::string>* currTrack = nullptr;
    std::string title;
    int choice = 0;

    do{
        std::cout << " \n ---------------------------\n";
        std::cout << "     CROSSROAD'S MUSIC PLAYER    ";
        std::cout << " \n ---------------------------\n";
        std::cout << " \n -------    MENU   ---------\n";
        std::cout << "1 - Play All\n";
        std::cout << "2 - Next Song\n";
        std::cout << "3 - Previous Song\n";
        std::cout << "4 - Add Song\n";
        std::cout << "5 - Remove Song\n";
        std::cout << "6 - DELETE PLAYLIST\n";

        std::cout << "0 - Exit\n";
        std::cin >> choice;
        std::cin.ignore();

        switch(choice) {
            case 1:
                std::cout << "----PLAYLIST----"<<std::endl;
                cdllTraversal(playlist);
                break;

            case 2:
                next(&currTrack);
                break;
```

```
37
38              case 3:
39                  previous(&currTrack);
40                  break;
41
42              case 4:
43                  std::cout << "----PLAYLIST----"<<std::endl;
44                  std::getline([&]std::cin, [&]title);
45                  cdllInsertEnd( title, &playlist);
46                  if (currTrack == nullptr) {
47                      currTrack = playlist;
48                  }
49                  std::cout << title <<" has been added"<<std::endl;
50                  break;
51
52              case 5:
53                  if (playlist == nullptr) {
54                      std::cout << "EMPTY PLAYLIST"<<std::endl;
55                      break;
56
57                  }
58                  std::cout << "Enter song title to remove: ";
59                  std::getline([&]std::cin, [&]title);
60                  cdllDeleteNode( title, &playlist);
61                  std::cout << title <<" has been removed"<<std::endl;
62
63                  if (playlist == nullptr) {
64                      currTrack = nullptr;
65                  }
66                  break;
```

```
            case 6:
                std::cout << "DELETING PLAYLIST"<<std::endl;
                cdllDeleteList(&playlist);
                currTrack = nullptr;
                break;

            case 0:
                std::cout << "Exiting!"<<std::endl;
                break;

            default:
                std::cout << "INVALID CHOICE"<<std::endl;
                break;
        }

    } while(choice != 0);


return 0;

}
```

Outputs:

```
     CROSSROAD'S MUSIC PLAYER
 --------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
4

----PLAYLIST----
Thunder

Thunder has been added

 --------------------------
     CROSSROAD'S MUSIC PLAYER
 --------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
4

----PLAYLIST----
Like a G6

Like a G6 has been added
```

```
 --------------------------
     CROSSROAD'S MUSIC PLAYER
 --------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
4

----PLAYLIST----
Black Velvet

Black Velvet has been added

 --------------------------
     CROSSROAD'S MUSIC PLAYER
 --------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
1

----PLAYLIST----
Thunder->Like a G6->Black Velvet->Playlist Restarted
```

```
 ---------------------------
     CROSSROAD'S MUSIC PLAYER
 ---------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
2

Like a G6 is now playing

 ---------------------------
     CROSSROAD'S MUSIC PLAYER
 ---------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
2

Black Velvet is now playing
```

```
 --------------------------
     CROSSROAD'S MUSIC PLAYER
 --------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
3

Like a G6 is now playing

 --------------------------
     CROSSROAD'S MUSIC PLAYER
 --------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
3

Thunder is now playing
```

```
 ---------------------------
     CROSSROAD'S MUSIC PLAYER
 ---------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
5

Enter song title to remove:Like a G6

Like a G6 has been removed

 ---------------------------
     CROSSROAD'S MUSIC PLAYER
 ---------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
1

----PLAYLIST----
Thunder->Black Velvet->Playlist Restarted
```

```
 --------------------------
     CROSSROAD'S MUSIC PLAYER
 --------------------------

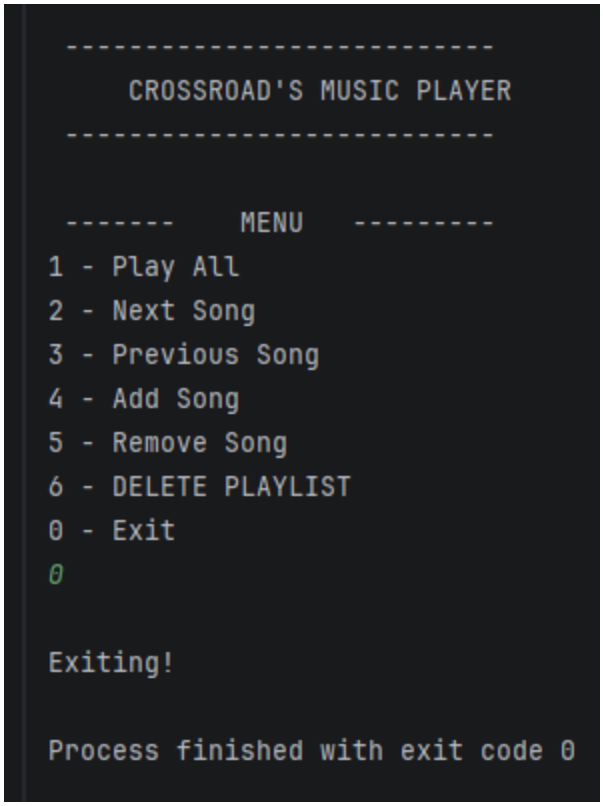
 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
6

DELETING PLAYLIST

 --------------------------
     CROSSROAD'S MUSIC PLAYER
 --------------------------

 -------    MENU   ---------
1 - Play All
2 - Next Song
3 - Previous Song
4 - Add Song
5 - Remove Song
6 - DELETE PLAYLIST
0 - Exit
1

----PLAYLIST----
Empty Playlist
```



Conclusion:

In today's activity I learned about the definition of linked lists. Moreover, I also learned how to create and implement a singly linked list. Furthermore, we also understood how linked lists work, and how it is being used or implemented. In connection to this, in the procedure section I made a singly and double linked list and both of them have the same functions but the doubly linked list uses two pointers rather than one for moving forwards and backwards. We also created different functions like traversal, wherein the program will display all the data in the linked list. Insertion at the head, wherein a new node will be inserted at the head which then will become the new head. Insertion at any part, wherein we can insert a node on the list. Insertion at the end, wherein we insert a new node at the end and it will become the new tail. Deletion of a node, wherein the program will delete a node that contains the data needed to be deleted. And lastly, delete all lists wherein the list will be deleted, and point to null. Next is the supplementary activity where I was tasked to create a program that is a music player wherein it will create a playlist and the user can input, play, remove, songs. And they can also go next, or set it to the previous song. On the contrary the user can also delete a song, and delete the playlist. In order to make it, I create a header file for a circular doubly linked list so that I can loop the music. In addition, I did a traverse function, wherein it plays all the songs. Two delete functions, wherein it deletes a song, and the other deletes the whole playlist. A next and previous function wherein it plays the next or previous song. And after the user is done using the player, they can exit. With that, I think this activity helped me understand what linked list really does and the supplementary part was very challenging, and although it was challenging I still completed it and I strive to be better and enhance my programming skill.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Concept explanation, and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI as a guide and explained to me what a code does.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.